FastZX BASIC 4.3 — User Manual

Oct 27, 2026 · @Mateusz

1. Getting started

FastZX BASIC is a fast, structured BASIC that runs on the ZX Spectrum 128K (and 48K). You write programs in a full-screen editor without line numbers, much like QBasic or FreeBASIC: it shows 51 columns, indents your code for you, and runs the program on the machine. Everything is integer arithmetic, which makes it many times faster than Sinclair BASIC. A machine-code game library adds fast text, colour, scrolling, masked sprites with pixel-exact collisions, tile maps and joystick input. Finished games can be saved as stand-alone tapes that load straight from the 128K Tape Loader. On a 128K, the extra memory banks add 64 KB for large arrays, maps and stored screens (section 4), and the AY sound chip plays music and sound effects in the background while your program runs (section 11).

Loading

1. Switch on the 128K and choose **Tape Loader** (or type `LOAD ""` in 48 BASIC).
2. Play `fastzx.tap` from a tape player, TZXDuino or an emulator.
3. After about a minute the editor appears: a blue screen with a status bar showing "Ln 1/1" and, on the bottom line, the banner and the free memory.

Your first program

Type these three lines, pressing ENTER at the end of each. The editor keeps your indentation:

```
FOR I = 1 TO 5
  PRINT "Hello, Spectrum! ";I
NEXT I
```

Now press **EDIT** (CAPS SHIFT + 1). The command prompt `>` appears. Type `RUN` and press ENTER. The program prints five lines, then "OK" and "Press any key". Press a key and you are back in the editor with your program.

When you move off a line, the editor stores it in a tidy form: keywords become capitals and spacing is made regular. `print "hi";x` becomes `PRINT "hi";X`.

What is on the tape

After the system itself, `fastzx.tap` holds nine example programs. Leave the tape running, press EDIT and type for example `LOAD "game"`:

Name	What it shows
`sieve`	Speed: counts the primes below 8000 and prints the time taken
`lines`	PLOT and DRAW line art
`guess`	INPUT, REPEAT/UNTIL, labels and BEEP in a number-guessing game
`procs`	Recursive procedures and functions: Towers of Hanoi, Fibonacci, GCD
`game`	A small game: joystick menu, coloured sprites, background music, explosion effect, scrolling stars
`sprites`	Eight masked, coloured sprites bouncing over a striped background
`screens`	128K: three full screens kept in extended memory and switched instantly with keys 1-3
`sound`	128K: background music (Ode to Joy) while the screen animates; keys 1-8 play the eight sound effects
`tiles`	A maze game built from tiles: a 48 x 20 map in extended memory, a scrolling view, walls that block, coins to collect (Q A O P to move)

Four stand-alone tapes are also supplied: `game.tap`, `sprites.tap`, `sound.tap` and `tiles.tap` load from the Tape Loader and start by themselves, without FastZX. Section 12 shows how to make your own.

Leaving and coming back

`BYE` at the command prompt returns to Sinclair BASIC. `RANDOMIZE USR 32771` comes back with your program intact. `RANDOMIZE USR 32768` restarts FastZX with an empty program.

2. The editor

The screen is a text editor, not a command line. Rows 1 to 22 show your program, the cyan row shows where the cursor is (`Ln 12/40 Col 5`, and `CAPS` when caps lock is on), and the bottom row is a short help line. There are no line numbers: a program is simply a list of lines, and the editor numbers them for you when it reports a position or an error.

Keys

Key	Action
letters, digits, SYMBOL SHIFT symbols	Insert at the cursor
ENTER	Split the line at the cursor. The new line starts with the same indentation
CAPS SHIFT + 0 (DELETE)	Delete the character left of the cursor. At the start of a line, join it to the line above
CAPS SHIFT + 5 / 8	Cursor left / right (moves to the previous / next line at the ends)
CAPS SHIFT + 6 / 7	Cursor down / up
CAPS SHIFT + 3 / 4	Page up / page down (21 lines)
CAPS SHIFT + 9	Delete the whole current line
CAPS SHIFT + 2	Caps lock on / off
CAPS SHIFT + 1 (EDIT)	Open the command prompt
SYMBOL SHIFT + Q / E / W	Type `<=`, `>=`, `<>`
SYMBOL SHIFT + Y / U / S	Type `AND`, `OR`, `NOT`
SYMBOL SHIFT + G / F / D / I / A	Type `THEN`, `TO`, `STEP`, `AT`, `STOP`

Lines can be up to 250 characters. A line longer than the screen scrolls sideways while you edit it; other lines show their first 32 characters.

51 or 32 columns

The editor starts in **51-column mode**: a small 4 x 7 font in 5-pixel steps shows 51 characters per line, so most program lines fit without scrolling sideways. Press **EXTEND** (CAPS SHIFT and SYMBOL SHIFT together) to switch to the classic 32-column display with the normal Spectrum font, and again to switch back. The choice is kept until you leave FastZX. The cursor in 51-column mode is an inverted character cell; the status line and the command prompt always use 32 columns.

(A true 52-column mode is not possible with equal-width characters: the screen is 256 pixels wide, and 256 / 52 is not a whole number. 51 columns of 5 pixels is the widest even layout, which is why Spectrum programs usually call it the "51-column" or "52-column" mode.)

The 51-column font is only for the editor. Programs print with the normal 32-column font.

Indentation and tidy lines

FastZX indents your program for you. You can type every line flush left: as soon as you leave a line (ENTER, cursor up or down), the editor works out each line's indentation from the program's structure:

- Lines inside FOR ... NEXT, WHILE ... WEND, REPEAT ... UNTIL, a block IF ... ENDIF, PROC ... ENDPROC and FUNC ... ENDFUNC move 2 spaces to the right for each level.
- NEXT, WEND, UNTIL, ENDIF, ENDPROC, ENDFUNC and ELSE line up with the line that opened the block.
- A line that follows a line ending in a comma (a DIM continued on the next line) gets 4 extra spaces.
- A loop that opens and closes on one line (`FOR I = 1 TO 3: PRINT I: NEXT I`) and the one-line IF do not change the indentation.

After ENTER the cursor is placed at the new line's indentation, ready to type. For example, typing these lines exactly as shown, without any spaces at the start:

```
proc show(n)
for i=1 to n
if i mod 2=0 then
print i;" even"
else
print i;" odd"
endif
next i
endproc
```

produces:

```
PROC SHOW(N)
  FOR I=1 TO N
    IF I MOD 2=0 THEN
      PRINT I;" even"
    ELSE
      PRINT I;" odd"
    ENDIF
  NEXT I
ENDPROC
```

Because the indentation always follows the structure, a line that is indented wrongly is a sign that a NEXT, WEND, UNTIL or ENDIF is missing or misplaced. Spaces you type at the start of a line are replaced by the computed indentation; the indentation is stored with the line (one byte) and costs no extra memory.

The command prompt

EDIT clears the screen and shows a `>` prompt. Type one of these, or any statement, and press ENTER:

Command	Effect
`RUN`	Clear the variables and run the program from the first line
`LIST`	Print the program on the screen (BREAK stops it)
`NEW`	Erase the program
`SAVE "name"`	Save the program to tape

Command	Effect
`SAVE "name" RUN`	Save a stand-alone game (section 12)
`LOAD "name"`	Load a program (`LOAD ""` loads the next one found)
`EDIT n`	Go back to the editor at line n
`BYE`	Return to Sinclair BASIC
`PRINT 2+2` , `X = 5` , …	Any statement runs immediately, using the program's variables

Pressing ENTER on an empty prompt goes straight back to the editor. After a command or program finishes, "OK" and "Press any key" appear; any key returns to the editor.

When something goes wrong

An error stops the program and prints a message such as `Division by zero in line 14` . Press a key and the editor opens with the cursor on line 14. BREAK (CAPS SHIFT + SPACE) stops a running program the same way.

Labels instead of line numbers

To jump, give a line a label: a name of two or more letters or digits followed by a colon at the start of the line. Then use `GOTO name` or `GOSUB name` :

```
N = 5
countdown:
  PRINT N;" ";
  N = N - 1
  IF N > 0 THEN GOTO countdown
PRINT "lift off!"
GOSUB done
END
done:
  PRINT "subroutine reached"
  RETURN
```

This prints `5 4 3 2 1 lift off!` and then `subroutine reached` . A label can share its line with statements (`mainloop: X = X + 1`). Names that are exactly a keyword, such as `END:` , are not labels. Most programs need no labels at all: WHILE, REPEAT, FOR, PROC and

FUNC (sections 5 and 6) are clearer.

3. Language basics

Statements and lines

Each line holds one or more statements separated by `:`. `REM` starts a comment that runs to the end of the line. Upper and lower case are the same, except inside quotes.

```
REM a comment
X = 10: Y = 20: PRINT X + Y
```

Numbers

All numbers are 16-bit signed integers from -32768 to 32767. Arithmetic wraps around silently, as on most 8-bit machines: `32767 + 1` gives `-32768` and `300 * 300` gives `24464`. Write hexadecimal numbers with `$`, e.g. `$FF` or `$C000`. Numbers above 32767 (up to 65535) are allowed and are stored as the same 16 bits, so `PEEK(65535)` and `POKE 50000, 1` work.

Variables

Kind	Names	Notes
Integer	`A` to `Z`, or a name	A single letter, or a name of 2 to 6 characters: a letter, then letters, digits or `_` (`SCORE`, `X1`). RUN sets them to 0
String	`A$` to `Z$`	Up to 63 characters each. RUN empties them
Array	`A(...)` to `Z(...)`	Made with DIM (section 4). Separate from the plain variable of the same letter

Named variables

Besides the 26 single letters, a variable can have a **name of 2 to 6 characters**: a letter, then letters, digits or `_`. This works for all three kinds:

Kind	Single letter	Named
Number	`S`	`SCORE`, `LIVES`, `X1`, `HI_SC`
String	`N$`	`NAME$`, `MSG$`
Array	`A()`	`HISCRS()`, `MAP2()`

```
DIM HISCRS(4) = 500, 400, 300, 200, 100
SCORE = 0: LEVEL = 1: NAME$ = "Ann"
FOR RANK = 0 TO 4
  PRINT RANK + 1; ". "; HISCRS(RANK)
NEXT RANK
PRINT NAME$; " is on level "; LEVEL
```

Things to know:

- `S`, `SC` and `SCORE` are three different variables, and a name can be a number, a string and an array at the same time (`SCORE`, `SCORE$` and `SCORE()` are separate), as with single letters.
- A name cannot be exactly a keyword (`PRINT`, `TIME`, `TO`, `IF` …), but it may *start* with one: `TOTAL`, `TIMER`, `NOTE`, `VALUE` and `TILES` are all fine names.
- **Put a space between a keyword and a name.** Up to 6 characters run together are read as one name, so `PRINTX` is a variable called PRINTX, and `FORI=1TO3` means a variable called FORI. Write `PRINT X` and `FOR I = 1 TO 3`. A keyword followed directly by digits still works (`TO10`, `STEP2`), and so does glued text longer than 6 characters (`PRINTSCORE` is still `PRINT SCORE`). Programs saved before keep working, and when you edit an old line, the editor lists it with spaces.
- Names have at most 6 characters. A longer word that is not a keyword gives `Syntax error`; procedures, functions and labels can still have long names.
- A program can use up to 112 different names in all (numbers, strings and arrays together), on top of the 78 single-letter variables. More gives the error `Too many names`.
- Memory for them is taken when the program runs, only for the names it uses: 2 bytes for each number, 64 for each string and 4 for each array (plus the array's elements, as with any array). Using `SCORE` is almost as fast as using `S`, because names are numbered once, when the program starts, not looked up while it runs.

- Like arrays, named variables are cleared when you change the program. After RUN you can use them in typed commands (`PRINT SCORE`); a name the program does not use gives `Unknown name`.
- Procedure and function parameters, LOCAL, FOR, INPUT, DIM, ADR, TILEMAP, SCRSAVE and SCRLOAD all accept names, and the compiler handles them too.

Strings

A string expression joins pieces with `+`. A piece is a literal `"text"`, a string variable, `CHR$(n)`, `STR$(n)`, `MID$(v$, start, count)` or `INKEY$`. Strings can be compared with `=` and `<>`.

```
N$ = "World": M$ = "Hello, " + N$ + CHR$(33)
PRINT M$;" has ";LEN(M$);" characters"
IF MID$(M$, 1, 5) = "Hello" THEN PRINT "starts with Hello"
PRINT "Number as text: " + STR$(-12) + ", and back: ";VAL("42") + 1
```

A quote mark inside a string is written `CHR$(34)`. While a string is being built it can be up to 255 characters; it is cut to 63 when stored in a variable.

Operators, highest priority first

Priority	Operators	Meaning
1	`( )`, functions	Grouping
2	`-`	Negation
3	`*` `/` `MOD`	Multiply, divide (rounds towards zero), remainder (takes the sign of the left value)
4	`+` `-`	Add, subtract
5	`=` `<>` `<` `>` `<=` `>=`	Compare (signed). The result is 1 for true, 0 for false
6	`NOT`	1 if the value is 0, otherwise 0
7	`AND`	Bitwise AND
8	`OR` `XOR`	Bitwise OR and exclusive OR

A condition is true when its value is not 0. Comparisons give exactly 1 or 0, so `IF X > 0 AND X < 10 THEN` works as expected.

Watch out with bit tests. `J AND 2` gives 2, not 1, when bit 1 is set. On its own (`IF J AND 2 THEN`) that is fine, because 2 counts as true. But combined with a comparison, `(J AND 2) AND X > 0` is `2 AND 1`, which is 0. Write `(J AND 2) > 0 AND X > 0` instead.

Watch out, too, that AND and OR always evaluate both sides. `IF I > 0 AND H(I - 1) < S THEN ...` still reads `H(-1)` when I is 0 and stops with `Bad subscript`. Split such tests: `D = 0: IF I > 0 THEN D = H(I - 1) < S`.

A quick test of both rules:

```
J = 16: H = 1
PRINT (J AND 16) AND H
PRINT (J AND 16) > 0 AND H > 0
```

prints `0` (16 AND 1 is 0) and then `1`.

Examples: `-7 / 2` is `-3`, `-7 MOD 2` is `-1`, `7 MOD 3` is `1`, `NOT 5` is `0`, `6 AND 3` is `2`, `6 OR 3` is `7`, `6 XOR 3` is `5`.

4. Variables, arrays and DIM

Plain assignment works as usual (`X = 5`, `N$ = "Bob"`, the word `LET` is optional). `DIM` creates arrays and can also give starting values to arrays, numbers and strings in one statement.

DIM forms

Form	Meaning
`DIM A(10)`	Array with indexes 0 to 10, all 0
`DIM A(4) = 12, 32, 23, 45, 12`	Array with starting values. Missing values are 0; too many is an error
`DIM A() = 7, 8, 9`	Size taken from the values: here indexes 0 to 2
`DIM A(3) = {1, 2, 3, 4}`	Curly brackets are allowed, as in FreeBASIC
`DIM G(15) BYTE = ...`	A BYTE array: each value takes one byte (for graphics and other data)

Form	Meaning
`DIM X = 5`	Sets a number variable
`DIM N$ = "Player 1"`	Sets a string variable
`DIM X = 5, N$ = "Bob", P() = 1, 2`	Several in one DIM, separated by commas

A DIM statement may continue on the next lines: end a line with a comma and carry on. Indent the continuation lines to make it clear:

```
DIM P(4) = 2, 3, 5, 7, 11
DIM S = 0, L = 3, N$ = "Player 1"
DIM M() = 10, 20, 30
DIM Z(9)
PRINT N$;" score ";S;" lives ";L
FOR I = 0 TO 4: PRINT P(I);" ";: NEXT I
PRINT
PRINT "M has ";M(2);" at index 2, Z(9)=";Z(9)
```

Output:

```
Player 1 score 0 lives 3
2 3 5 7 11
M has 30 at index 2, Z(9)=0
```

Arrays

Arrays have one dimension and start at index 0, so `DIM A(10)` has 11 elements. Every access is checked: an index outside the array stops the program with `Bad subscript`. Arrays are cleared by RUN and whenever the program is edited, so create them at the start of the program.

BYTE arrays for data

A BYTE array stores each value in one byte, which is the natural form for sprite graphics, UDGs, maps and tables. Read single bytes with `PEEK(ADR(G) + i)` and pass `ADR(G)` to statements that need an address:

```
DIM G() BYTE = 24, 60, 126, 255,
               255, 126, 60, 24
FOR I = 0 TO 7
  PRINT PEEK(ADR(G) + I);" ";
NEXT I
```

This prints `24 60 126 255 255 126 60 24`. Reading `G(i)` directly gives two bytes at a time (bytes 2i and 2i+1 as one number), which is rarely what you want.

In other words BYTE only changes how the starting values are packed; `A(i)` on a BYTE array still reads and writes 16-bit elements, each made of two bytes:

```
DIM A() BYTE = 1, 2, 3, 4
PRINT A(0); " "; A(1)
PRINT PEEK(ADR(A)); " "; PEEK(ADR(A) + 3)
```

prints `513 1027` (1 + 2 x 256 and 3 + 4 x 256) and `1 4`. A BYTE array of n+1 bytes made with `DIM A(n) BYTE` has elements A(0) to A(n/2). Tile maps (section 10) are BYTE arrays too: read and change them with GETTILE and SETTILE, which work in bytes and also work for FAR maps.

Useful tricks

- **Lookup tables** replace slow calculations: `DIM Q() = 0, 1, 4, 9, 16, 25` and then `Q(X)` instead of `X*X`.
- **UDGs**: copy a BYTE array into the UDG area (`DPEEK(23675)` is its address) with a FOR loop and POKE, then draw it with `PUTCH col, row, 144`.
- **Strings in data**: `ADR(N$)` is the address of the text of N$ for PEEK and POKE.

FAR arrays: the 128K's extra memory

Normally your program and its arrays share about 18 KB. On a 128K, add `FAR` to a DIM and the array goes into the extra memory banks instead: 64 KB that nothing else uses. A FAR array is used exactly like any other array; FastZX switches the memory banks for you on every access.

```
DIM M(25000) FAR
DIM T() BYTE FAR = 3, 192, 15, 240
DIM L(99) FAR = 1, 2, 3
PRINT FARFREE;" words of extended memory left"
```

- `FAR` can be combined with everything DIM offers: sizes, BYTE, starting values, `()` auto size and continuation lines.
- `FARFREE` gives the room left in extended memory, counted in 2-byte array elements (32767 when empty).
- All FAR arrays together can use up to 65534 bytes, e.g. one word array of 32766 elements or several smaller ones.
- `FRE` still reports the normal memory, which FAR arrays leave free for program lines.

Good uses are level maps, large lookup tables, high-score lists, the moves of a puzzle, and stored screens (section 8).

The trade-offs:

- A FAR array has no normal address, so `ADR()` gives the error `FAR array has no address`. Sprite graphics, UDGs, POKES and machine code must stay in normal arrays.
- An access is slightly slower: about 10% in the interpreter and up to 50% in compiled code, where the array access itself is most of the work.
- On a 48K Spectrum, `FAR` is ignored and the arrays go into normal memory, so the same program still runs if it fits.
- FastZX uses memory banks 1, 3, 4 and 6, which 128 BASIC uses for its RAM disk (`SAVE !`). Files in the RAM disk are lost. Of bank 7 it uses only the second screen area (C000-DAFF), which the ROMs do not use, for the music notes and the tune compiler; 128 BASIC's own data in bank 7 lies above it, so `BYE` returns safely to 128 BASIC.

5. Program flow

IF

The one-line form runs the rest of the line only when the condition is true:

```
IF X > 10 THEN PRINT "big": X = 10
```

The block form ends its first line with `THEN` and finishes with `ENDIF`. `ELSE` is optional, and blocks can be nested:

```
FOR T = 5 TO 35 STEP 10
  IF T < 10 THEN
    PRINT T;": cold"
  ELSE
    IF T < 30 THEN
      PRINT T;": nice"
    ELSE
      PRINT T;": hot"
    ENDIF
  ENDIF
NEXT T
```

This prints `5: cold`, `15: nice`, `25: nice`, `35: hot`. The one-line form has no ELSE; use the block form when you need one.

FOR ... NEXT

`FOR V = start TO limit [STEP s]` counts V from start to limit. The limit and step are worked out once at the start. If the loop would not run even once (for example `FOR I = 5 TO 1`), its body is skipped entirely. `NEXT` may name the variable (`NEXT I`) or not.

```
FOR I = 10 TO 0 STEP -2: PRINT I;" ";: NEXT I
```

prints `10 8 6 4 2 0`.

WHILE ... WEND and REPEAT ... UNTIL

`WHILE` tests at the top and may run zero times; `REPEAT` tests at the bottom and always runs at least once:

```
N = 1
WHILE N < 100
  N = N * 2
WEND
PRINT "first power of 2 above 100: ";N
K = 0
REPEAT
  K = K + 7
UNTIL K MOD 5 = 0
PRINT "first multiple of 7 and 5: ";K
```

This prints 128 and 35. A typical game is one big `REPEAT ... UNTIL` loop with `PAUSE 1` at the top (section 9).

GOTO, GOSUB and labels

`GOTO label` jumps to the line that starts with `label:`. `GOSUB label` does the same but comes back when it meets `RETURN`. GOTO and GOSUB also accept a number or expression, meaning that editor line, but labels are much easier to keep right. Prefer procedures (section 6) to GOSUB in new programs.

Ending

`END` (or `STOP`) finishes the program. Running past the last line also ends it.

6. Procedures and functions

A procedure is a named block of code you call with `EXEC`. A function is the same, but it gives back a value and is called with `FN` inside an expression. Names can be any length (letters, digits and `_`), parameters are local to the call, and both may call themselves.

```
EXEC BANNER(3)
PRINT "5! = ";FN FACT(5)
PRINT "max = ";FN MAX(17, 42)
END

PROC BANNER(N)
  LOCAL I
  FOR I = 1 TO N
    PRINT "*** ";I;" ***"
  NEXT I
ENDPROC

FUNC FACT(N)
  IF N < 2 THEN RETURN 1
  RETURN N * FN FACT(N - 1)
ENDFUNC

FUNC MAX(A, B)
  IF A > B THEN RETURN A
  RETURN B
ENDFUNC
```

Output: three banner lines, then `5! = 120` and `max = 42`.

Statement	Meaning
`PROC name [(p1, p2, ...)]` … `ENDPROC`	Defines a procedure. PROC must be the first statement of its line
`FUNC name [(p1, p2, ...)]` … `ENDFUNC`	Defines a function. FUNC must be the first statement of its line
`EXEC name [(e1, e2, ...)]`	Calls a procedure (or a function, ignoring the result)
`FN name [(e1, e2, ...)]`	Calls a function inside an expression
`RETURN [e]`	Leaves the procedure or function at once; in a function e is the result
`LOCAL v [, v ...]`	Makes more variables local, starting at 0. Put it straight after PROC/FUNC

How parameters work

Variables are still the global letters, but while a procedure runs, its parameter and LOCAL variables have their own values. When the procedure ends, the old values come back. In the example, BANNER's `I` does not disturb any `I` used by the main program. All other variables are shared, which is a simple way to return extra results:

```
EXEC DIVIDE(17, 5)
PRINT Q;" remainder ";R
END
PROC DIVIDE(A, B)
  Q = A / B: R = A MOD B
ENDPROC
```

Rules

- Up to 8 parameters. The number of values in a call must match the definition.
- Parameters and results are integers. Strings and arrays are shared globals.
- A function that reaches ENDFUNC without `RETURN value` gives 0.

- When the program runs into a PROC or FUNC line, the whole definition is skipped, so definitions can go anywhere (usually after END).
- To leave early, use `RETURN`, not ENDPROC.
- Recursion is limited only by memory; very deep recursion stops with `Out of memory` instead of crashing.

7. Keyboard, joystick and INPUT

Four ways to read the keyboard

Function	Use it for
`INPUT "prompt"; v`	Typing a number or a line of text (v or v$). Waits for ENTER
`GETKEY`	Waiting for one key press; gives its code
`INKEY` / `INKEY$`	Checking for a key without waiting; 0 or "" if none since the last check
`KEY("Q")`	Whether a key is held down right now. Several keys can be held at once, which is what games need

```
INPUT "Your name";N$
INPUT "Your age";A
PRINT "Hello ";N$;", in ten years you will be ";A + 10
```

`KEY` takes the key's character: `KEY("A")`, `KEY("7")`, `KEY(" ")` for SPACE, `KEY(CHR$(13))` for ENTER, `KEY(CHR$(1))` for CAPS SHIFT and `KEY(CHR$(2))` for SYMBOL SHIFT. `GETKEY` and `INKEY` give lower-case letter codes unless CAPS SHIFT or caps lock is used.

Joysticks

`JOYSTICK n` chooses the controller, and `JOY` then reads it. JOY always gives the same bit pattern, whichever controller is used, so a game written for one works with all of them:

n	Controller	Keys it reads
0	Kempston (the default)	Port 31
1	Sinclair 1 (Interface 2 left, the +2's port 1)	6 left, 7 right, 8 down, 9 up, 0 fire
2	Sinclair 2 (Interface 2 right, the +2's port 2)	1 left, 2 right, 3 down, 4 up, 5 fire
3	Cursor / Protek	5 left, 6 down, 7 up, 8 right, 0 fire
4	Keyboard	Q up, A down, O left, P right, SPACE fire

JOY bit	Value	Direction
0	1	Right
1	2	Left
2	4	Down
3	8	Up
4	16	Fire

```
JOYSTICK 2
PRINT "Sinclair 2: use keys 1-5"
REPEAT
  J = JOY
  PRINT AT 5,0;"R";J AND 1;" L";(J AND 2)/2;" D";(J AND 4)/4;" U";(J AND
8)/8;" F";(J AND 16)/16
UNTIL J AND 16
```

In a game, test one direction at a time and remember the bit-test rule from section 3:

```
J = JOY
IF (J AND 2) > 0 AND X > 0 THEN X = X - 2
IF (J AND 1) > 0 AND X < 240 THEN X = X + 2
IF J AND 16 THEN EXEC FIRE
```

A good habit is to let the player choose: show a menu, read a digit with `GETKEY`, and pass it to JOYSTICK. The game demo does exactly this. Without a Kempston interface attached, port 31 may read as random values, so do not leave type 0 selected on a machine without one.

8. Text, colours and screen graphics

Two kinds of text output

`PRINT` works like Sinclair BASIC: it scrolls, understands `AT row,col`, `TAB col`, `;` and `,`, and uses the current INK and PAPER. It is convenient but goes through the ROM. The game statements `PUTCH` and `PUTS` write characters straight into screen memory: they are much faster, never scroll and do not change the colours, which makes them ideal for scores and status lines.

```
PRINT AT 0,0;"Score: ";S
PUTS 0, 0, "Score: " + STR$(S) + "   "
```

Both lines show the score in the top-left corner. Note that PRINT AT takes the row first, while all game statements take the column first, then the row.

Coordinates

Used by	Units	Range	Origin
PUTCH, PUTS, SETATTR, GETATTR, FILLATTR, FILLBOX, SCROLL, SCROLLC	Character cells (col, row)	col 0-31, row 0-23	Top left
SPRITE, SPRPOS	Pixels (x, y)	x 0-255, y 0-191 minus the sprite size	Top left
PLOT, DRAW, POINT	Pixels (x, y)	x 0-255, y 0-175	Bottom left, as in Sinclair BASIC
PRINT AT	Character cells (row, col)	row 0-21, col 0-31	Top left

Colours

`INK`, `PAPER`, `BRIGHT` and `BORDER` set the colours for PRINT and CLS (0 black, 1 blue, 2 red, 3 magenta, 4 green, 5 cyan, 6 yellow, 7 white). The cell statements use a whole attribute value instead:

`attribute = INK + 8 * PAPER + 64 * BRIGHT + 128 * FLASH`

For example, bright yellow on blue is 6 + 8 + 64 = 78, and black on cyan is 40.

Text and attribute statements

Statement / function	Effect
`PUTCH col, row, code`	Draws one character. Codes 144-164 are the UDGs
`PUTS col, row, s$`	Draws a string, stopping at column 31
`FONT addr`	Uses your own font: character c is read from addr + 8*c (like the CHARS system variable)
`SETATTR col, row, attr`	Sets one cell's colour
`GETATTR(col, row)`	Reads one cell's colour (0 outside the screen)
`FILLATTR c1, r1, c2, r2, attr`	Colours the rectangle from (c1, r1) to (c2, r2)
`FILLBOX c1, r1, c2, r2, byte`	Fills every pixel row of the rectangle with byte: 0 clears it, 255 makes it solid, 85 or 170 make stripes

A window with a title bar, drawn by a procedure:

```
BORDER 1: PAPER 0: INK 7: CLS
EXEC WINDOW(4, 5, 27, 12, 79)
PUTS 6, 7, "FastZX window"
PUTS 6, 9, "Press any key"
K = GETKEY
END

PROC WINDOW(A, B, C, D, E)
  FILLBOX A, B, C, D, 0
  FILLATTR A, B, C, D, E
  FILLBOX A, B, C, B, 255
  FILLATTR A, B, C, B, 112
ENDPROC
```

Scrolling

Statement	Effect
`SCROLL dir, c1, r1, c2, r2`	Moves the pixels of the rectangle by 1 pixel. Colours stay in place; blank pixels come in
`SCROLLC dir, c1, r1, c2, r2`	Moves the rectangle by one character cell, pixels and colours. The colour that comes in is the current PAPER/INK

Directions: 0 left, 1 right, 2 up, 3 down. A smoothly scrolling message:

```
CLS
PUTS 0, 10, "  Scrolling text - press a key  "
REPEAT
  SCROLL 0, 0, 10, 31, 10
  PAUSE 1
UNTIL INKEY <> 0
```

The cost grows with the area: one character row takes about 3 ms, the whole screen about 60 ms. Scroll only the part that needs it.

Lines and points

`PLOT x, y` sets a pixel, `DRAW dx, dy` draws a line from the last point, and `POINT(x, y)` reads a pixel. Here y = 0 is the bottom row, as in Sinclair BASIC. Points and lines that would leave the screen are ignored instead of causing an error.

Storing whole screens

`SCRSAVE A` copies the whole screen (pixels and colours, 6912 bytes) into array A, and `SCRLOAD A` puts it back instantly. The array must hold at least 6912 bytes: `DIM A(6911) BYTE` or, better, `DIM A(6911) BYTE FAR`, so the screens use the 128K's extra memory. Up to nine screens fit there.

```
DIM P(6911) BYTE FAR, Q(6911) BYTE FAR
EXEC TITLE: SCRSAVE P
EXEC LEVEL: SCRSAVE Q
SCRLOAD P: K = GETKEY
SCRLOAD Q
```

Use it for title and menu screens, for redrawing a level background in an instant, or to save the screen before showing a message and restore it afterwards. The `screens` demo on the tape draws three screens and switches between them with keys 1-3. The border colour is not part of the screen and is not stored.

9. Sprites

There are 8 sprites, numbered 0 to 7. Each is 8 x 8, 8 x 16, 16 x 8 or 16 x 16 pixels (width x height) and can be placed at any pixel position. A sprite is drawn in one of two ways:

- **XOR sprites** invert the pixels they cover. They are simple and slightly faster, but look scrambled over a busy background.
- **Masked sprites** cut a clean hole in the background with a mask, then draw the graphic. The background is saved and put back exactly when the sprite moves.

Either kind can also be given a colour with `SPRCOL`.

Sprite statements

Statement / function	Effect
`SPRDEF n, addr [, w, h]`	Makes sprite n an XOR sprite using the graphic at addr. w and h are 8 or 16 (default 16, 16)
`SPRMASK n, addr [, w, h]`	Makes sprite n a masked sprite using the mask and graphic at addr
`SPRCOL n, c`	Gives sprite n a colour: c = 0-7 sets the INK of the cells it covers, 8-15 the same but BRIGHT, -1 switches colour off. The original colours are put back when it moves
`SPRITE n, x, y`	Moves sprite n to pixel x, y and redraws it at once
`SPRPOS n, x, y`	Sets where sprite n should be, without drawing yet
`SPRDRAW`	Redraws every sprite at its new position in one go
`SPROFF n`	Hides sprite n
`HIT(a, b)`	1 if sprites a and b are both shown and their rectangles overlap

x can be 0 to 256 minus the width and y 0 to 192 minus the height; y = 0 is the top. A position outside that range hides the sprite. Defining a new graphic for a visible sprite redraws it in place, which is how you animate.

Graphic data

Rows go from top to bottom; bit 7 of each byte is the leftmost pixel. A BYTE array (section 4) is the natural place for the data.

Kind	Each row holds	Bytes for 8x8 / 8x16 / 16x8 / 16x16
XOR	the graphic: 1 byte if 8 wide, 2 bytes if 16 wide	8 / 16 / 16 / 32
Masked	the mask byte(s), then the graphic byte(s)	16 / 32 / 32 / 64

In the mask, 1 means the pixel belongs to the sprite (background hidden) and 0 means transparent. A mask one pixel larger than the graphic all round gives the sprite a neat black outline.

Example: an XOR sprite crossing the screen

```
DIM S() BYTE = 24, 60, 126, 219, 255, 36, 90, 165
SPRDEF 0, ADR(S), 8, 8
FOR X = 0 TO 240 STEP 2
  PAUSE 1
  SPRITE 0, X, 80
NEXT X
SPROFF 0
```

Example: four masked, coloured sprites

```
BORDER 0: PAPER 1: INK 7: CLS
FOR R = 0 TO 23 STEP 2: FILLBOX 0, R, 31, R, 85: NEXT R
REM 8x8 masked: each row = mask byte, graphic byte
DIM B() BYTE = 60,0, 126,60, 255,126, 255,90, 255,126, 255,102, 126,60, 60,0
FOR N = 0 TO 3
  SPRMASK N, ADR(B), 8, 8
  SPRCOL N, 10 + N
NEXT N
FOR T = 0 TO 200
  FOR N = 0 TO 3
    SPRPOS N, 20 + T + N * 12, 40 + N * 24
  NEXT N
  PAUSE 1: SPRDRAW
NEXT T
```

The four faces slide across the striped background without damaging it, each in its own bright colour.

SPRITE or SPRPOS + SPRDRAW?

`SPRITE` is simplest when sprites never overlap. When they can overlap, set every position with `SPRPOS` and then call `SPRDRAW` once per frame. SPRDRAW redraws the sprites one at a time, starting at the top of the screen: each sprite is erased and at once drawn at its new position, so it is missing from the screen only for that moment. Sprites whose old or new positions overlap are handled together as a group (erased 7 down to 0, then drawn 0 up to 7), so each saved background stays clean and overlapping sprites stack correctly, with sprite 7 on top. Use the same method when sprites have colours, so that their colour cells are restored in the right order.

Colour and attribute clash

The Spectrum has one INK and one PAPER per 8 x 8 cell. A coloured sprite therefore changes the INK of whole cells, including any background pixels in them: that is the familiar Spectrum colour clash. It looks best on a plain or dark background. On a pattern, like the sprites demo, the pattern takes on the sprite's colour around it, which can also be used as an effect.

Choose sprite colours that differ from the PAPER under them. SPRCOL sets only the INK; if it is the same colour as the paper (for example colour 1 or 9, blue, on a blue background), the sprite is drawn but cannot be seen, and only its bright or dim square shows. On a blue background use colours 10-15 (bright red, magenta, green, cyan, yellow, white); the sprites demo uses `SPRCOL N, 10 + N MOD 6` for exactly that reason.

Speed

Erasing and redrawing one sprite takes, depending on its size, its x position (a multiple of 8 is quickest) and colour: about 2-3 ms for an 8 x 8 sprite and 3.5-5.5 ms for a 16 x 16 one. A SPRDRAW of the 8 sprites in the `sprites` demo (four 16 x 16 and four 8 x 8, masked and coloured) takes about 27 ms, more than one 20 ms frame. Put `PAUSE 1` immediately before SPRDRAW at the top of each game loop (`PAUSE 1: SPRDRAW`), so the redraw starts just after the frame begins, while the TV picture is still being drawn at the top of the screen. With 3 or 4 sprites everything moves every frame; with 8, every other frame.

Flicker. A sprite flickers when the TV picture is drawn over it in the instant between its erase and its redraw. SPRDRAW keeps that instant as short as possible and works from the top down, so the picture usually reaches a sprite after it has been redrawn. Measured on the `sprites` demo, about 1 sprite drawing in 9 is still caught; with eight 8 x 8 sprites about 1 in 20. To reduce flicker further:

- use fewer or smaller sprites: an 8 x 8 sprite takes half the time of a 16 x 16 one;
- leave out SPRCOL on sprites that do not need colour (it adds about 10 percent);
- move sprites in steps of 8 pixels where it does not matter (unshifted sprites are about a third faster);
- keep `PAUSE 1: SPRDRAW` together, with the program's other work after SPRDRAW, not between PAUSE and SPRDRAW.

Collisions

Three functions test whether sprites touch. All use the sprites' new positions (from SPRITE or SPRPOS), so you can test a move before drawing it, and all ignore hidden sprites.

Function	Result	Cost
`HIT(a, b)`	1 if the rectangles of sprites a and b overlap	very fast

Function	Result	Cost
`SPRHIT(n)`	Number of the first other sprite whose rectangle overlaps sprite n, or -1	fast: up to 7 rectangle tests
`PIXHIT(a, b)`	1 if a and b overlap in at least one solid pixel	fast when apart (the rectangle test comes first); about one row test per overlapping pixel row

The rectangle test is generous: two round 16 x 16 sprites "hit" when only their transparent corners overlap. PIXHIT compares the actual shapes: for a masked sprite it uses the mask (every pixel that hides the background counts as solid), for an XOR sprite its graphic. A typical game uses SPRHIT or HIT every frame and confirms with PIXHIT only when that says yes, or simply uses PIXHIT, which does the cheap rectangle test itself first.

```
CLS
DIM S() BYTE = 24,60,126,255,255,126,60,24
FOR I = 0 TO 3: SPRDEF I, ADR(S), 8, 8: NEXT I
SPRITE 0, 100, 100
SPRITE 1, 40, 40: SPRITE 2, 104, 96: SPRITE 3, 200, 20
H = SPRHIT(0)
IF H >= 0 THEN PRINT "Sprite 0 touches sprite "; H
PRINT "Box test: "; HIT(0, 2); "  pixel test: "; PIXHIT(0, 2)
SPRITE 2, 106, 94
PRINT "Corners only: "; HIT(0, 2); "  pixel test: "; PIXHIT(0, 2)
```

Output:

```
Sprite 0 touches sprite 2
Box test: 1  pixel test: 1
Corners only: 1  pixel test: 0
```

The diamond-shaped sprites overlap clearly at first; after sprite 2 moves up and right only the empty corners of the two boxes overlap, so HIT still says 1 but PIXHIT says 0. To test a sprite against walls and other parts of the background, use tile maps (section 10).

10. Tile maps

Many games are built from a grid of 8 x 8 blocks: walls, floors, ladders, coins, water. FastZX keeps such a level as a **map**, one byte per cell holding a **tile number**, and the tile engine draws it, scrolls it and tells you which tiles a sprite touches. All of it is machine code, so a full-screen map draws in about a tenth of a second.

Tile types: TILE

`TILE n, addr [, attr [, flags]]` defines tile type n (0-63):

Part	Meaning	Default
`addr`	Address of 8 bytes of graphic, top row first, bit 7 = left pixel (like a sprite or UDG). Usually `ADR(G) + 8 * k` for the k-th tile in a BYTE array	
`attr`	Colour of the cell: ink + 8 x paper (+ 64 bright)	the current INK and PAPER
`flags`	A number whose bits you give a meaning, for TILEHIT	0

A common choice of flags is 1 = solid (walls, floors), 2 = collectable (coins, keys), 4 = deadly (spikes, water), 8 = ladder. A tile can have several: 1 + 4 for a wall that kills. Tile types that were never defined are blank (and tile 0 is usually left as the empty space). RUN clears all tile types.

The map: TILEMAP

The map is a BYTE array with one byte per cell, row by row: cell (x, y) is byte `y * width + x`. Give it starting values with DIM, or make an empty one of the right size and fill it with SETTILE:

```
DIM M() BYTE = 1,1,1,1, 1,0,2,1, 1,1,1,1
TILEMAP M, 4, 3

DIM L(48 * 20 - 1) BYTE FAR
TILEMAP L, 48, 20
```

`TILEMAP v, width, height` tells the engine which array is the map and how wide and high it is; it stops with `Bad subscript` if the array does not exist or is smaller than width x height bytes. Large maps belong in extended memory (`FAR`): a 128K holds maps of up to 64 KB without using any of the normal memory. Only one map is in use at a time; a game with several levels keeps each in its own array and calls TILEMAP to switch.

Remember that `M(i)` on a BYTE array reads two bytes at once (section 4). Read and change map cells with GETTILE and SETTILE.

Building a map from text: MAPKEY, MAPROW

The easiest way to fill a map is to draw it as text, one string per row, and let FastZX translate each character into a tile:

Command	What it does
`MAPKEY k$`	Character number n of k$ (counting from 0) stands for tile n. Up to 47 characters
`MAPROW y, s$`	Puts the characters of s$ into map row y, starting at column 0, translated by MAPKEY. Characters not in the key become tile 0; a row longer than the map is cut off; a row outside the map is ignored
`MAPCOUNT(n)`	The number of map cells holding tile n
`MAPFIND(n)`	1 if some cell holds tile n, else 0. TILEX and TILEY then give the first such cell, row by row

```
DIM M(10 * 4 - 1) BYTE
TILEMAP M, 10, 4
MAPKEY ".#*P"
MAPROW 0, "##########"
MAPROW 1, "#P..*..*.#"
MAPROW 2, "#..####..#"
MAPROW 3, "##########"
N = MAPCOUNT(2)
IF MAPFIND(3) THEN X = TILEX: Y = TILEY: SETTILE X, Y, 0
```

With the key `".#*P"`, `.` is tile 0 (empty), `#` tile 1, `*` tile 2 and `P` tile 3. N becomes 2 (two `*`), and MAPFIND finds the `P` at column 1, row 1, which is then cleared: a tile number you never define with TILE works as an invisible marker for a starting position. All four

commands work on normal and FAR maps, run in machine code (a 48 x 20 cave is built in a fraction of a second), and are also available in compiled programs.

Showing the map: SHOWMAP

`SHOWMAP mapx, mapy, col, row, width, height` draws part of the map in a rectangle of the screen (the **view**): map cell (mapx, mapy) appears at screen cell (col, row), and width x height cells are drawn. The view is clipped at the screen edges; cells outside the map are drawn as blank.

```
SHOWMAP 0, 0, 0, 1, 32, 20
```

shows the top-left 32 x 20 cells of the map from screen row 1 down, leaving row 0 free for a score. **Scrolling** is simply SHOWMAP again with a different mapx or mapy: `SHOWMAP V, 0, 0, 1, 32, 20` with V increased by 8 shows the map 8 cells further right. A full-screen view takes about a tenth of a second to draw (a little more for a FAR map), so scroll in larger steps, when the player nears the edge of the view, rather than every frame. Smaller views draw proportionally faster.

SHOWMAP draws over everything in the view, including sprites; the sprites are drawn afresh, at their new positions, by the next SPRITE or SPRDRAW, so call that straight after SHOWMAP.

Reading and changing the map

Statement / function	Effect
`GETTILE(x, y)`	Tile number in map cell (x, y), or -1 outside the map
`SETTILE x, y, n`	Puts tile n in map cell (x, y); if that cell is in the view, it is redrawn at once
`TILEAT(x, y)`	Tile under screen pixel (x, y) (sprite coordinates: 0-255 across, 0-191 down from the top), or -1 if that point is outside the view or the map
`TILEX`, `TILEY`	Map cell of the tile found by the last TILEAT or TILEHIT

SETTILE works before any SHOWMAP too, which is how a program builds a level:

```
FOR X = 0 TO W - 1: SETTILE X, 0, 1: SETTILE X, H - 1, 1: NEXT X
```

If a masked sprite stands on the cell you change, hide it first (`SPROFF n: SETTILE ...`) and draw it again afterwards; otherwise, when it moves, it would put back the old background it saved, including the tile you just removed.

Sprites and tiles: TILEHIT

`TILEHIT(n, flags)` looks at every map cell under sprite n (a 16 x 16 sprite covers up to 9 cells) and gives the number of the first tile whose flags share a bit with `flags`, or -1 if there is none. TILEX and TILEY then hold that tile's map cell. Like the sprite collision functions it uses the sprite's new position, so the usual pattern is: **try the move, test it, keep it only if it is free.**

```
SPRPOS 0, X + E, Y
IF TILEHIT(0, 1) < 0 THEN X = X + E
```

Here E is the step left or right and flag 1 means solid: the sprite only moves when the new position touches no wall. Testing the horizontal and vertical steps separately lets the player slide along walls. Collecting things uses another flag:

```
IF TILEHIT(0, 2) >= 0 THEN
  SPROFF 0: SETTILE TILEX, TILEY, 0
  C = C + 1: SFX 3
ENDIF
```

Example: a small map

```
CLS
DIM G() BYTE = 255,129,129,129,129,129,129,255, 0,24,60,126,126,60,24,0
TILE 1, ADR(G), 1*8+5, 1
TILE 2, ADR(G)+8, 6, 2
DIM M() BYTE = 1,1,1,1,1,1,1,1, 1,0,0,2,0,0,0,1, 1,0,1,1,1,0,2,1,
1,1,1,1,1,1,1,1
TILEMAP M, 8, 4
SHOWMAP 0, 0, 12, 10, 8, 4
PRINT AT 0,0;"Tile at map 3,1: ";GETTILE(3, 1)
PRINT "Tile under pixel 124,90: ";TILEAT(124, 90)
```

Tile 1 is a cyan-on-blue wall with flag 1, tile 2 a yellow coin with flag 2. The 8 x 4 map is drawn with its top-left corner at screen cell (12, 10). Both lines print 2: map cell (3, 1) holds a coin, and pixel (124, 90) is in screen cell (15, 11), which shows map cell (3, 1).

The tiles demo

`LOAD "tiles"` (or tiles.tap) is a complete maze game in 70 lines. Worth reading for:

- the level is generated at random into a 48 x 20 FAR map with SETTILE, counting the coins as it goes;
- the view is 32 x 20 cells; when the player gets within 4 cells of the right or left edge, the view moves 8 cells (`V = V + 8: EXEC VIEW`, where PROC VIEW calls SHOWMAP);
- the player is in map pixels (X, Y) and drawn at `X - V * 8, Y + 8` on the screen;
- walls block with the try-test-keep pattern above, coins are removed with SPROFF, SETTILE and a sound effect.

Tips

- Plan the flags first. With flags you can change the meaning of a tile (a door that opens) by redefining it with TILE, without touching the map.
- Keep tile graphics in one BYTE array, 8 bytes per tile, and define the tiles with `TILE k, ADR(G) + 8 * (k - 1), ...` in a FOR loop.
- Tiles are aligned to character cells, so their colours never clash with each other. Sprites moving over coloured tiles take on the cell colours (section 9); give the sprites SPRCOL colours that go with the tiles.
- The engine's limits: 64 tile types and one map at a time; a map can be as large as its array allows (up to 64 KB in FAR memory).

11. Sound

The beeper

`BEEP cycles, period` plays a tone on the beeper through the ROM routine. The program waits while it plays.

- period = 437500 / frequency - 30
- cycles = frequency x seconds

A C major scale, with the periods kept in an initialised array:

```
DIM N() = 1642, 1460, 1297, 1223, 1086, 964, 856, 806
FOR I = 0 TO 7
  BEEP 60, N(I)
NEXT I
```

Note	C	D	E	F	G	A	B	C
period	1642	1460	1297	1223	1086	964	856	806

The AY chip (128K)

`SOUND register, value` writes one register of the 128K's AY-3-8912 sound chip. It plays on its own while the program continues, so it suits game music and effects. The most useful registers:

Register	Purpose
0, 1	Channel A tone period, low and high byte (2, 3 for B; 4, 5 for C)
6	Noise period (0-31)
7	Mixer: 62 = only tone A on, 56 = tones A, B and C on, 55 = noise A on
8, 9, 10	Volume of channels A, B, C (0-15)

The tone period for a frequency f is about 110840 / f (440 Hz is 252). A falling "laser" sound:

```
SOUND 7, 62: SOUND 8, 15
FOR P = 400 TO 100 STEP -4
  SOUND 0, P MOD 256: SOUND 1, P / 256
  PAUSE 1
NEXT P
SOUND 8, 0
```

Always end with `SOUND 8, 0` (volume 0), or the tone keeps playing after the program stops.

Background music: MUSIC

`MUSIC` plays a tune on the AY chip *while your program keeps running*: an interrupt routine advances the music 50 times a second, so PAUSE, game loops, INPUT and everything else carry on as normal. Tunes are written in the same notation as QBasic's `PLAY`. Each string is one channel, so up to three voices play together:

```
MUSIC "T120 O3 L4 E E F G G F E D C C D E E. D8 D2", "T120 O1 L1 V10 C G C L2 G C"
```

Statement / function	Effect
`MUSIC a$ [, b$ [, c$]]`	Plays a$ on channel A, b$ on B and c$ on C, once, in the background
`MUSIC REPEAT a$ [, b$ [, c$]]`	The same, repeating forever (ideal for game music)
`MUSIC STOP`	Stops the music and silences it
`PLAYING`	1 while music is playing, 0 when it has finished

Starting a new MUSIC replaces the old one. The music also stops when the program ends or stops with an error.

The tune notation

Command	Meaning	Default
`C D E F G A B`	Play a note. Follow with `#` or `+` for sharp, `-` for flat, then optionally a length and dots: `C#8`, `B-4.`	
`R` or `P`	Rest (silence), with optional length: `R4`, `P8`	
`L n`	Default note length: 1 whole, 2 half, 4 quarter, 8 eighth, 16, 32, 64	L4
`.` after a length	Dotted: one and a half times as long	
`O n`	Octave 0-7. O3 starts at middle C (as in QBasic)	O4
`< >`	One octave down / up	
`T n`	Tempo in quarter notes per minute, 32-255	T120

Command	Meaning	Default
`V n`	Volume 0-15	V12
`MN` `ML` `MS`	Normal (7/8 of the length sounds), legato (all of it), staccato (3/4)	MN

Spaces are ignored, so space the notes as you like. Each MUSIC string can be up to 255 characters (a literal string, or several joined with `+`; remember a string variable holds only 63). Each channel takes up to about 100 notes per MUSIC statement. Timing is in 1/50 s steps, so very fast notes are rounded.

A two-voice example in a procedure, so the game can restart it easily:

```
PROC TUNE
  MUSIC REPEAT "T160 O2 L8 V9 C C G G A A G4 F F E E D D C4", "T160 O1 L2 V8
C C F C G C"
ENDPROC
```

To play tunes longer than one statement allows, wait for `PLAYING` to become 0 and start the next part, e.g. inside the game loop: `IF PLAYING = 0 THEN EXEC PARTTWO`.

Sound effects: SFX

`SFX` plays a short sound effect, also in the background. Effects use channel C: while an effect plays, music on channel C is muted and channels A and B continue. So keep game music to two voices if you use effects.

Statement	Effect
`SFX n`	Plays built-in effect n (0-7)
`SFX s$`	Plays your own effect, described by a string
`SFX STOP`	Stops the current effect

n	Built-in effect	Its description
0	Shot / laser	`P60 S14 L14 D1`
1	Explosion	`N24 T0 L32 D2`

n	Built-in effect	Its description
2	Jump	`P500 S-25 L14 V13`
3	Coin / pick-up	`P140 S-6 L10 V14`
4	Hit / hurt	`P200 S30 L16 D1 N8`
5	Power-up	`P420 S-12 L30 V12`
6	Menu blip	`P120 L4 V12`
7	Bounce	`P300 S-40 L7 V14`

Your own effect is a string of letters with numbers:

Letter	Meaning	Default
`P n`	Starting tone period (1-4095); smaller is higher. Middle C is 424	200
`S n`	Change of the period every frame, may be negative: `S-20` rises in pitch, `S20` falls	0
`L n`	Length in frames (50 = 1 second)	10
`V n`	Starting volume 0-15	15
`D n`	Volume drops by 1 every n frames (0 = no fade)	0
`N n`	Add noise with noise period n (0-31; small = hiss, large = rumble)	off
`T0`	No tone, noise only	tone on

```
SFX "P80 S-4 L20 V15 D2"
SFX "N5 T0 L6 V12"
```

The first is a rising zap that fades out; the second a short hiss, good for footsteps or a sword swing. The `sound` demo plays all eight built-in effects on keys 1-8 while music runs, which is the quickest way to hear them.

Notes on the sound system

- MUSIC and SFX need the 128K's AY chip (also in the +2, +2A and +3). On a 48K they are silently ignored, so a program still runs. A stand-alone game made with `SAVE "name" RUN` on a 48K has no tune compiler on its tape, so when it is loaded on a 128K it runs without music; save music games on a 128K.
- The music routine changes the Z80's interrupt mode while FastZX runs and puts it back on BYE. Your own machine code (CALL, USR) must not change the I register or the interrupt mode.
- `SOUND` still writes AY registers directly. While MUSIC or SFX play they will overwrite the registers they use, so use SOUND only when the players are silent.
- `BEEP` uses the beeper and stops the program while it plays; the AY music carries on.

12. Saving, loading and stand-alone games

Command	Effect
`SAVE "name"`	Saves the program to tape, for loading back into FastZX
`LOAD "name"`	Loads a program; other files on the tape are skipped. `LOAD ""` loads the next one
`SAVE "name" RUN`	Saves a stand-alone game: a BASIC loader plus FastZX and your program in one block

SAVE prints "Start tape, then press any key". Start recording first, then press a key. Names can be up to 10 characters.

Stand-alone games

A tape made with `SAVE "mygame" RUN` loads from the 128K Tape Loader (or `LOAD ""` on a 48K) and starts your program at once, as if you had typed RUN. Nobody needs FastZX to play it. It contains:

Tape file	Contents
Program: mygame	`CLEAR 24575: LOAD "" CODE: LOAD "" CODE: RANDOMIZE USR 32777`, starting at line 10

Tape file	Contents
Bytes: mygame	The FastZX runtime (about 19 KB, starting at 7300 hex, followed by a 1 KB block with the tune compiler, which loads into the screen and is moved to bank 7 at start-up) and your program

If the game ends with END or an error, the FastZX editor appears with the program, so it can be listed or fixed. To return to Sinclair BASIC instead, finish with `BYE`. Most games simply loop forever, or go back to their title screen.

13. Optional: the PC compiler

`fastzxc.py` (in the source archive) compiles a FastZX program into Z80 machine code on a PC with Python 3. The result is a stand-alone tape for the Tape Loader, like `SAVE "name" RUN`, but it runs typically 4 to 20 times faster: loops, conditions, arrays and procedure calls become native code. The graphics, sprite and joystick statements call the same machine-code library as the interpreter.

```
python3 fastzxc.py mygame.bas        (a text file: one program line per text
line)
python3 fastzxc.py mygame.tap        (a program saved with SAVE "name" on the
Spectrum)
```

Options: `-o name.tap` sets the output file, `--nobreak` removes the BREAK checks, `--nolines` drops line numbers from error messages, and `--asm file` also writes the generated Z80 assembly.

The text file is written exactly as in the editor: no line numbers, indentation allowed, labels with `name:`. The compiled program gives the same results as the interpreter (checked by running thousands of random expressions through both), with a few stricter rules:

- STEP in FOR must be a constant, and each FOR must have exactly one NEXT written after it.
- RUN, LIST, NEW, EDIT, SAVE and LOAD cannot be used.
- FN cannot be used inside string expressions or statements such as INPUT and BEEP; assign it to a variable first.
- Errors print their message, wait for a key and return to Sinclair BASIC.

14. A complete game, step by step

This is the `game` program from the tape (also supplied as `game.tap`). You steer a yellow ball and dodge a bouncing red one, while stars scroll along the bottom of the screen. It uses most of what the previous sections describe.

1. Title and controls menu

```
REM FastZX 4.0 demo: joystick, coloured sprites, scrolling
BORDER 0: PAPER 0: INK 7: CLS
PRINT AT 8,5;"Choose your controls:"
PRINT AT 10,5;"1  Kempston joystick"
PRINT AT 11,5;"2  Sinclair 1 (keys 6-0)"
PRINT AT 12,5;"3  Sinclair 2 (keys 1-5)"
PRINT AT 13,5;"4  Cursor keys (5-8, 0)"
PRINT AT 14,5;"5  Q A O P + SPACE"
REPEAT: K = GETKEY - 49: UNTIL K >= 0 AND K <= 4
JOYSTICK K
CLS
```

GETKEY gives the code of the key pressed; the digit "1" has code 49, so `GETKEY - 49` turns keys 1-5 into 0-4, exactly the numbers JOYSTICK expects. The REPEAT loop ignores any other key.

2. Graphics and screen layout

```
DIM S() BYTE = 3,192,15,240,31,248,59,220,127,254,127,254,255,255,255,255,
    255,255,255,255,127,254,127,254,59,220,31,248,15,240,3,192
DIM E() BYTE = 128,1,64,2,35,196,23,232,15,240,31,248,63,252,255,255,
    255,255,63,252,31,248,15,240,23,232,35,196,64,2,128,1
FILLATTR 0,0,31,1,79: PUTS 0,0," FASTZX 4.0  dodge the red one "
FILLATTR 0,22,31,23,5
SPRDEF 0, ADR(S): SPRCOL 0, 14
SPRDEF 1, ADR(E): SPRCOL 1, 10
```

The two 16 x 16 XOR sprites are 32 bytes each, given as BYTE arrays that continue onto a second line. Rows 0-1 become a bright title bar (attribute 79 = bright white on blue) and rows 22-23 cyan, for the stars. SPRCOL makes sprite 0 bright yellow (14) and sprite 1 bright red (10).

3. The game loop

```
X = 40: Y = 100: A = 180: B = 60: U = 2: V = 1: L = 3: P = 0
REPEAT
  PAUSE 1
  J = JOY
  IF (J AND 2) > 0 AND X > 0 THEN X = X - 2
  IF (J AND 1) > 0 AND X < 240 THEN X = X + 2
  IF (J AND 8) > 0 AND Y > 16 THEN Y = Y - 2
  IF (J AND 4) > 0 AND Y < 160 THEN Y = Y + 2
  A = A + U: IF A < 1 OR A > 239 THEN U = -U
  B = B + V: IF B < 17 OR B > 159 THEN V = -V
  SPRITE 0, X, Y: SPRITE 1, A, B
  EXEC STARS
  P = P + 1
  IF P MOD 25 = 0 THEN PUTS 1,1,"SCORE " + STR$(P / 25) + "  LIVES " +
STR$(L) + " "
  IF HIT(0, 1) THEN EXEC OUCH
UNTIL L = 0
SPROFF 0: SPROFF 1: PUTS 11,11,"GAME OVER"
END
```

X, Y is the player, A, B the enemy and U, V its speed; L counts lives and P frames. Each pass:

1. `PAUSE 1` waits for the next frame, so the game runs at a steady 50 steps per second.
2. `JOY` is read once and each direction bit is tested (note the `> 0` in each test, see section 3). The limits keep the player on the screen and below the title bar.
3. The enemy moves and bounces by reversing its speed at the edges.
4. Both sprites are redrawn; they are XOR sprites, so overlapping briefly is harmless.
5. Every 25 frames (half a second) the score line is redrawn with PUTS, which is fast and does not scroll.
6. HIT checks for a collision.

4. Procedures

```
PROC STARS
  SCROLL 0, 0,22, 31,23
  IF RND(4) = 0 THEN PLOT 255, RND(16)
ENDPROC

PROC OUCH
  LOCAL I
  FOR I = 1 TO 6: BORDER I: BEEP 10, 300 + I * 50: NEXT I
  BORDER 0
  L = L - 1: A = 200: B = 40
ENDPROC
```

STARS scrolls the bottom two rows one pixel left and sometimes plots a new star at the right edge (PLOT counts y from the bottom, so 0-15 are rows 22-23). OUCH flashes the border with rising beeps, takes a life and moves the enemy away. `LOCAL I` keeps the loop counter from disturbing anything else.

Ideas to try

- Turn the enemy into a masked sprite with `SPRMASK` and draw a background behind the game.
- Add a second enemy (sprite 2) and use `SPRPOS` / `SPRDRAW`.
- Make fire (`J AND 16`) shoot an 8 x 8 bullet sprite, and use `HIT` between bullet and enemy.
- Save your version with `SAVE "mygame" RUN` and give the tape to a friend.

15. Cookbook

Short, complete programs for things games often need. Each one has been run as printed. Type them in (the editor indents them for you), or load them from the `examples` folder of the source package.

Jumping and gravity

A figure that walks with O and P, jumps with SPACE and lands on platforms. The level is a tile map; flag 1 means solid.

```
REM Platform jumping: gravity, floors from the tile map
BORDER 0: PAPER 0: INK 7: CLS
DIM G() BYTE = 255,255,219,255,255,219,255,255
TILE 1, ADR(G), 2*8+6, 1
DIM M(32*22-1) BYTE
TILEMAP M, 32, 22
FOR X = 0 TO 31: SETTILE X, 21, 1: NEXT X
FOR Y = 0 TO 20: SETTILE 0, Y, 1: SETTILE 31, Y, 1: NEXT Y
FOR X = 6 TO 12: SETTILE X, 18, 1: NEXT X
FOR X = 15 TO 21: SETTILE X, 15, 1: NEXT X
SHOWMAP 0, 0, 0, 0, 32, 22
DIM P() BYTE = 60,126,90,126,60,24,60,102
SPRDEF 0, ADR(P), 8, 8: SPRCOL 0, 7
JOYSTICK 4
X = 16: Y = 100: V = 0
REPEAT
  PAUSE 1
  J = JOY
  E = 2 * ((J AND 1) > 0) - 2 * ((J AND 2) > 0)
  IF E <> 0 THEN
    SPRPOS 0, X + E, Y
    IF TILEHIT(0, 1) < 0 THEN X = X + E
  ENDIF
  V = V + 1: IF V > 6 THEN V = 6
  SPRPOS 0, X, Y + V
  IF TILEHIT(0, 1) >= 0 THEN
    IF V > 0 THEN Y = TILEY * 8 - 8
    V = 0
  ELSE
    Y = Y + V
  ENDIF
  SPRPOS 0, X, Y + 1
  IF (J AND 16) > 0 AND TILEHIT(0, 1) >= 0 THEN V = -8: SFX 2
  SPRITE 0, X, Y
UNTIL KEY("x")
```

How it works:

- V is the vertical speed in pixels per frame. Each frame it grows by 1 (gravity), up to 6 (falling speed limit).

- The vertical move is tried with SPRPOS and TILEHIT. If the new position is in a wall and the figure was falling, it is placed on top of that tile: `TILEY * 8 - 8` is the pixel row just above it (the sprite is 8 high). Hitting anything stops the movement (V = 0), so a head bump ends a jump.
- "Standing" means: one pixel lower would be inside a tile. Only then does fire start a jump (V = -8, a 28-pixel jump, enough to reach the next platform).
- `E = 2 * ((J AND 1) > 0) - 2 * ((J AND 2) > 0)` turns the right and left bits into a step of 2, -2 or 0 in one line.

Shooting

A ship at the bottom (O, P, SPACE to fire), an enemy moving across the top, pixel-exact hits.

```
REM Shooting: a bullet sprite and pixel-exact hits
BORDER 0: PAPER 0: INK 7: CLS
DIM S() BYTE = 24,24,60,126,255,255,36,36
DIM E() BYTE = 129,66,60,90,255,60,66,129
DIM B() BYTE = 24,24,24,24,0,0,0,0
SPRDEF 0, ADR(S), 8, 8: SPRDEF 1, ADR(E), 8, 8: SPRDEF 2, ADR(B), 8, 8
SPRCOL 0, 5: SPRCOL 1, 2: SPRCOL 2, 6
JOYSTICK 4
X = 120: A = 0: D = 2: Q = -1: R = 0: P = 0
REPEAT
  PAUSE 1
  J = JOY
  IF (J AND 1) > 0 AND X < 240 THEN X = X + 3
  IF (J AND 2) > 0 AND X > 0 THEN X = X - 3
  IF (J AND 16) > 0 AND Q < 0 THEN Q = X: R = 168: SFX 0
  A = A + D: IF A < 1 OR A > 239 THEN D = -D
  SPRPOS 0, X, 176: SPRPOS 1, A, 24
  IF Q >= 0 THEN
    R = R - 6
    IF R < 0 THEN
      Q = -1: SPROFF 2
    ELSE
      SPRPOS 2, Q, R
    ENDIF
  ENDIF
  SPRDRAW
  IF Q >= 0 THEN
```

```
    IF PIXHIT(2, 1) THEN
      SFX 1: P = P + 10: Q = -1: SPROFF 2: A = RND(200) + 20
      PUTS 0, 0, "Score " + STR$(P)
    ENDIF
  ENDIF
UNTIL KEY("x")
```

Q is the bullet's column, or -1 when no bullet is flying (only one at a time); R is its row. All three sprites are moved with SPRPOS and drawn together by one SPRDRAW, which keeps the movement smooth. A hit plays the explosion, adds 10 points and restarts the enemy at a random place.

A menu

```
REM A menu: Q and A move, ENTER chooses
CLS
PRINT AT 5,10;"Start game"
PRINT AT 7,10;"Options"
PRINT AT 9,10;"Quit"
C = 0
REPEAT
  FILLATTR 9, 5 + C * 2, 22, 5 + C * 2, 5 * 8
  K = GETKEY
  FILLATTR 9, 5 + C * 2, 22, 5 + C * 2, 7 * 8
  IF K = 97 AND C < 2 THEN C = C + 1: SFX 6
  IF K = 113 AND C > 0 THEN C = C - 1: SFX 6
UNTIL K = 13
PRINT AT 15,0;"You chose item "; C + 1
```

The chosen line is highlighted by colouring its cells (cyan paper) and put back to white paper before the cursor moves. GETKEY waits for a key and gives its code: 97 is a, 113 is q, 13 is ENTER. Pressing A, A, Q, A, ENTER chooses item 3.

Measuring time

```
REM Reaction timer: TIME counts frames (50 a second)
CLS
PRINT "Press a key when the border turns red"
PAUSE 50 + RND(100)
BORDER 2: T = TIME
K = GETKEY
BORDER 7
PRINT "Your time: "; (TIME - T) * 20; " ms"
```

TIME counts 50 frames a second, so one frame is 20 ms. The difference of two readings is correct even when the counter wraps around, as long as less than about 10 minutes pass.

A high-score table

```
REM Keeping the five best scores in order
DIM H() = 500, 400, 300, 200, 100
EXEC ADDSCORE(350)
EXEC ADDSCORE(50)
EXEC ADDSCORE(900)
FOR I = 0 TO 4: PRINT I + 1; ". "; H(I): NEXT I
END

PROC ADDSCORE(S)
  LOCAL I, D
  IF S <= H(4) THEN RETURN
  I = 4
  REPEAT
    D = 0
    IF I > 0 THEN D = H(I - 1) < S
    IF D THEN H(I) = H(I - 1): I = I - 1
  UNTIL D = 0
  H(I) = S
ENDPROC
```

Output: `1. 900`, `2. 500`, `3. 400`, `4. 350`, `5. 300`. A new score moves the lower ones down one place until it reaches its spot. Note `IF I > 0 THEN D = ...`: writing `I > 0 AND H(I - 1) < S` would read H(-1) (section 3).

Common pitfalls

Symptom	Cause and cure
A test with AND is never true	AND works on bits: compare first, `(J AND 16) > 0 AND ...` (section 3)
`Bad subscript` from a guarded test	AND and OR always evaluate both sides; split the test into two IFs
`Syntax error` in `SCORE = 0`	A name is at most 6 characters and cannot be exactly a keyword; put a space between a keyword and a name (`PRINT SCORE`)
Odd numbers from a BYTE array	`A(i)` reads two bytes; use PEEK(ADR(A) + i), or GETTILE for maps
Ghost images of a removed tile	Hide a masked sprite (SPROFF) before changing the tile under it
The game runs at uneven speed	Start the main loop with `PAUSE 1` so each turn starts on a new frame
Sprites flicker	Move them with SPRPOS and draw them all with one SPRDRAW
The music stops	MUSIC stops at END and errors; use MUSIC REPEAT for a tune that loops

16. Statement reference (A-Z)

In the syntax, `e` is any number expression, `s$` any string expression and `v` a variable; square brackets mark optional parts.

Statement	Description	Section
`BEEP cycles, period`	Beeper tone (stops the program while it plays)	11
`BORDER e`	Border colour 0-7	8
`BRIGHT e`	1 = bright on, 0 = off, for PRINT and CLS	8

Statement	Description	Section
`BYE`	Return to Sinclair BASIC	1
`CALL addr`	Call machine code (must keep IY, end with RET)	19
`CLS`	Clear the screen with the current colours; hides all sprites	8
`DIM ...`	Arrays and starting values (all forms in section 4); `FAR` puts an array in the 128K's extra memory	4
`DPOKE addr, e`	Write a 16-bit value (low byte first)	3
`DRAW dx, dy`	Line from the last plotted point (ignored if it leaves the screen)	8
`EDIT n`	(command) back to the editor at line n	2
`END` / `STOP`	End the program	5
`EXEC name [(e, ...)]`	Call a procedure	6
`FILLATTR c1, r1, c2, r2, attr`	Colour a rectangle of cells	8
`FILLBOX c1, r1, c2, r2, byte`	Fill the pixels of a rectangle of cells	8
`FONT addr`	Use your own character set	8
`FOR v = e TO e [STEP e]` … `NEXT [v]`	Counting loop	5
`FUNC name [(v, ...)]` … `ENDFUNC`	Define a function	6
`GOSUB label` / `RETURN`	Call a subroutine	5
`GOTO label`	Jump to a label (or a line number)	5
`IF c THEN statements`	One-line condition	5
`IF c THEN` … `[ELSE]` … `ENDIF`	Block condition	5
`INK e` / `PAPER e`	Colours for PRINT and CLS	8
`INPUT ["prompt";] v [, v ...]`	Read numbers or text	7

Statement	Description	Section
`JOYSTICK n`	Choose controller: 0 Kempston, 1 Sinclair 1, 2 Sinclair 2, 3 cursor, 4 QAOP	7
`name:`	A label at the start of a line	2
`[LET] v = e` / `v$ = s$` / `v(e) = e`	Assignment	3
`LIST`	(command) print the program	2
`LOAD "name"`	Load a program	12
`LOCAL v [, v ...]`	Local variables in a PROC/FUNC	6
`MUSIC [REPEAT] a$ [, b$ [, c$]]` / `MUSIC STOP`	Background music on up to 3 channels (128K)	11
`NEW`	(command) erase the program	2
`OUT port, e`	Write to an I/O port	3
`PAUSE e`	Wait e frames (50 = 1 second); `PAUSE 1` syncs a game loop	9
`PLOT x, y`	Set a pixel (y = 0 at the bottom)	8
`POKE addr, e`	Write a byte	3
`POKES addr, b, b, ...`	Write several bytes	4
`PRINT [items]`	Print with `;`, `,`, `AT row,col`, `TAB col`	8
`PROC name [(v, ...)]` … `ENDPROC`	Define a procedure	6
`PUTCH col, row, code`	Draw a character directly	8
`PUTS col, row, s$`	Draw a string directly	8
`RANDOMIZE [e]`	Seed the random numbers	17
`REM text`	Comment	3
`REPEAT` … `UNTIL c`	Loop tested at the end	5
`RETURN [e]`	Leave a GOSUB, PROC or FUNC (with a result)	5, 6
`RUN`	(command) run the program	2

Statement	Description	Section
`SAVE "name" [RUN]`	Save; with RUN as a stand-alone game	12
`SCRLOAD v`	Put back a screen stored with SCRSAVE	8
`SCROLL dir, c1, r1, c2, r2`	Scroll a rectangle by 1 pixel (0 left, 1 right, 2 up, 3 down)	8
`SCROLLC dir, c1, r1, c2, r2`	Scroll a rectangle by one character, with colours	8
`SCRSAVE v`	Copy the whole screen into array v (at least 6912 bytes, FAR allowed)	8
`SETATTR col, row, attr`	Colour one cell	8
`SETTILE x, y, n`	Put tile n in map cell (x, y); redraws it if it is in the view	10
`SFX n` / `SFX s$` / `SFX STOP`	Sound effect in the background (128K)	11
`SHOWMAP mapx, mapy, col, row, width, height`	Draw part of the tile map on the screen (the view)	10
`SOUND reg, value`	Write an AY sound register (128K)	11
`SPRCOL n, c`	Sprite colour (0-15, -1 off)	9
`SPRDEF n, addr [, w, h]`	Define an XOR sprite	9
`SPRDRAW`	Redraw all sprites at their new positions	9
`SPRITE n, x, y`	Move and draw a sprite	9
`SPRMASK n, addr [, w, h]`	Define a masked sprite	9
`SPROFF n`	Hide a sprite	9
`SPRPOS n, x, y`	Set a sprite's position for the next SPRDRAW (and for collision tests)	9
`TILE n, addr [, attr [, flags]]`	Define tile type n (0-63)	10
`TILEMAP v, width, height`	Use BYTE array v (normal or FAR) as the tile map	10

Statement	Description	Section
`WHILE c` … `WEND`	Loop tested at the start	5

Building maps from text (section 10): `MAPKEY k$` sets which character stands for which tile, and `MAPROW y, s$` fills map row y from a string. The matching functions are `MAPCOUNT(n)` (cells holding tile n) and `MAPFIND(n)` (finds the first one; TILEX and TILEY give its cell).

17. Function reference (A-Z)

`INKEY`, `INKEY$`, `GETKEY`, `JOY`, `TIME`, `FRE`, `FARFREE`, `PLAYING`, `TILEX` and `TILEY` take no brackets; all other functions need them.

Function	Result
`ABS(e)`	Absolute value
`ADR(v)` / `ADR(v$)`	Address of an array's data / of a string variable's text (not for FAR arrays)
`ASC(s$)`	Code of the first character (0 for an empty string)
`CHR$(e)`	One-character string
`DPEEK(addr)`	16-bit value at an address
`FARFREE`	Room left in the 128K's extra memory, in 2-byte array elements (0 on a 48K)
`FN name [(e, ...)]`	Result of a function (section 6)
`FRE`	Free bytes of normal memory for program lines, arrays and the stack
`GETATTR(col, row)`	Colour of a cell
`GETKEY`	Waits for a key and gives its code
`GETTILE(x, y)`	Tile in map cell (x, y), or -1 outside the map
`HIT(a, b)`	1 if the rectangles of sprites a and b overlap
`IN(port)`	Byte read from an I/O port

Function	Result
`INKEY`	Code of the key pressed since the last check, or 0
`INKEY$`	The same as a one-character string, or ""
`JOY`	Joystick bits: 1 right, 2 left, 4 down, 8 up, 16 fire
`KEY(s$)`	1 while that key is held down
`LEN(s$)`	Length of a string
`MID$(v$, start, count)`	Part of a string variable (start counts from 1)
`PEEK(addr)`	Byte at an address
`PIXHIT(a, b)`	1 if sprites a and b overlap in at least one solid pixel
`PLAYING`	1 while MUSIC is playing, 0 when it has finished
`POINT(x, y)`	1 if the pixel is set
`RND(n)`	Random number 0 to n-1; `RND(0)` gives 0 to 32767
`SGN(e)`	-1, 0 or 1
`SPRHIT(n)`	First other sprite whose rectangle overlaps sprite n, or -1
`STR$(e)`	Number as text
`TILEAT(x, y)`	Tile under screen pixel (x, y) through the view, or -1
`TILEHIT(n, flags)`	First tile under sprite n whose flags share a bit with flags, or -1
`TILEX`, `TILEY`	Map cell of the last TILEAT / TILEHIT result
`TIME`	The 50 Hz frame counter; the difference of two readings measures time
`USR(addr)`	Calls machine code and gives the value of the BC register
`VAL(s$)`	The number in a string (0 if it is not a number)

RANDOMIZE with no value seeds RND from the frame counter, so each run differs; `RANDOMIZE 7` gives the same sequence every time, which helps when testing.

18. Error messages

An error stops the program and prints the message with `in line n`, the editor line. Press a key and the editor opens at that line. Variables keep their values, so you can inspect them from the command prompt, e.g. `PRINT X`.

Message	Typical cause	What to do
Syntax error	Misspelled keyword, missing `=`, `,` or `)`, a long variable name such as SCORE	Check the line; a keyword needs a space before a name (`PRINT X`, not `PRINTX`), and names have at most 6 characters
Line not found	GOTO or GOSUB to a line number that does not exist	Use labels instead of numbers
Label not found	GOTO or GOSUB to a label that no line starts with	Check the spelling and the colon after the label
Out of memory	Program and arrays too large, recursion or nesting too deep, or extended memory full	Move big arrays to FAR, use BYTE arrays for data, fewer nested calls
Division by zero	`/` or `MOD` by 0	Test the divisor first
RETURN without GOSUB	RETURN reached outside any GOSUB, PROC or FUNC	Put END before the subroutines
NEXT without FOR	NEXT with no open FOR, or the wrong variable	Match each FOR with its NEXT
WEND without WHILE	WEND with no open WHILE	Match WHILE and WEND
UNTIL without REPEAT	UNTIL with no open REPEAT	Match REPEAT and UNTIL
Type mismatch	A string where a number is needed (or the reverse), a string parameter, or a string compared with `<` or `>`	Use STR$ / VAL to convert
String too complex	String expressions nested more than 4 deep	Split into steps with string variables

Message	Typical cause	What to do
Bad subscript	Array index out of range, array not created with DIM, too many DIM values, or an array too small for SCRSAVE	Check the DIM size and loop limits
BREAK	CAPS SHIFT + SPACE pressed	
Missing ENDIF/NEXT/WEND	A skipped block has no end before the end of the program	Close every IF, FOR and WHILE block
Line too long	A line is longer than 255 bytes after tidying	Split it into two lines
Tape loading error	LOAD failed; the program in memory is cleared	Rewind and try again; check the volume
ROM error X	A ROM routine reported Sinclair error X, e.g. `PRINT AT 30,0` gives ROM error 5 (out of screen)	Keep PRINT AT within rows 0-21
Unknown PROC/FUNC	EXEC or FN with a name that is not defined	Check the name; PROC/FUNC must start its line
Wrong number of arguments	A call passes a different number of values than the definition has parameters	Match the call to the definition
LOCAL outside PROC/FUNC	LOCAL outside a procedure, or after a loop was opened inside it	Put LOCAL straight after PROC/FUNC
FAR array has no address	ADR used on a FAR array	Keep data for sprites, UDGs, POKES and machine code in normal arrays

Two messages belong to named variables (section 4): **Unknown name**: a command typed at the prompt uses a name that the program does not use, so there is no such variable; RUN the program first, or check the spelling. **Too many names**: the program uses more than 112 different names; use single letters or arrays for some of them.

19. Technical notes

Why it is fast

- 16-bit integer arithmetic instead of the ROM's 5-byte floating point.
- Lines are tokenized once, when you leave them in the editor: keywords become single bytes and numbers are stored in binary.
- Keywords are dispatched through a jump table; simple expressions (a single number or variable) take a short cut.
- Variables live at fixed addresses, found with one shift instead of a search.
- FOR, WHILE, REPEAT, GOSUB and procedure calls keep their return points on the Z80 stack, so loops jump straight back without searching for lines.
- Graphics, sprites, scrolling and the joystick are machine-code routines.

Speed

Estimated from instruction counts in an emulator (treat them as a guide and time your own programs with `TIME`):

Test	Interpreter	Compiled (section 13)
Empty FOR/NEXT loop, 1000 times	about 0.3 s	about 0.04 s
`A = A + I*2 - B/3`, 1000 times	about 1.4 s	about 0.4 s
Array fill, 1000 elements	about 0.8 s	about 0.12 s
Procedure call with one parameter, 500 times	about 0.7 s	about 0.04 s
Sieve of primes up to 2000	about 10 s	about 0.8 s

The same empty loop takes several seconds in Sinclair BASIC.

FAR arrays cost a little extra per access, because FastZX switches the memory bank in and out each time: writing and reading 1001 elements takes about 1.2 s in normal memory and 1.3 s in FAR memory when interpreted, and about 0.23 s and 0.35 s when compiled.

Version 4.3 adds three interpreter speed-ups, each measured by counting Z80 instructions on benchmark programs: a shortcut for the most common expressions (a single number or variable, or `a op b` with two of them, such as `X + 1` or `I < N`), which skips the full expression parser; multiplication and division that take fewer steps for small

numbers; and a shorter NEXT. Loops, arithmetic, array work and procedure calls run 2 to 8 percent faster; long conditions such as `I MOD 7 = 0 AND I > 10` are about 4 percent slower, because the shortcut has to look ahead before handing them on. A faster general expression parser was tried and measured slower, so it was not kept. Two later changes help game loops in particular: a REM line inside the program now costs almost nothing (FastZX jumps straight to the next line instead of reading the comment), and EXEC and FN remember where each call in the program found its procedure, so calling the same procedure again skips the search. A platform-game loop with three procedures and comments ran about 18 percent faster with these two changes. For large gains, compile (section 13).

Tile engine speed: SHOWMAP draws a full 32 x 22 view in about 0.1 second (about 0.14 second for a FAR map); TILEHIT, TILEAT, GETTILE and SETTILE take well under a millisecond each.

Limits

Item	Limit
Number variables, string variables, arrays	26 single-letter of each (A-Z), plus up to 112 named variables (2-6 characters) in all
String variable length	63 characters (255 while an expression is built)
Program line	250 characters as typed, 255 bytes tokenized
Procedures and functions	40, with up to 8 parameters each
Labels	40
Sprites	8
Tile types / maps	64 tile types, one map in use at a time
Music	3 channels, about 100 notes per channel per MUSIC statement
Normal memory for program, normal arrays and stack	about 14.5 KB
Extended memory for FAR arrays and maps (128K only)	65534 bytes, e.g. 32766 word elements or nine stored screens

Memory map

Address (hex)	Contents
4000-5AFF	The screen (the tune compiler is loaded here from tape, then moved to bank 7)
5CCB-5FFF	Sinclair BASIC loader and system stack (CLEAR 24575)
6000-60FF	Input line; the line being edited
6100-62FF	Tokenized line (also runs commands)
6300-697F	The 26 string variables, 64 bytes each

Address (hex)	Contents
6A00-6DFF	String work buffers; the editor's display buffers
6E00-6E3F	Number, character, file name and tape header buffers
6E70-6F7F	Procedure table, argument buffers, library, editor (51-column) and extended-memory variables
6F80-6FCF	Sprite table (8 x 10 bytes)
7000-717F	Background saved under masked sprites (8 x 48 bytes)
7180-71C7	Colours saved under coloured sprites (8 x 9 bytes)
71D0-721F	Label table
7220-72FF	Transfer buffer for extended memory
7300-73FF	Operator table (loaded from tape)
7400-74FF	Note being played; stack for code running in bank 7
7500-7A5F	Keyword table and messages (loaded from tape)
7A60-7BDF	The editor's 51-column font (loaded from tape)
7C00-7CFF	Tile types (64 x 4 bytes)
7D00-7EFF	Statement and function dispatch tables (loaded from tape)
7F00-7FFF	Music, effect, tile-engine and SPRDRAW work variables; interrupt jump at 7F7F
8000-BBFF	FastZX code that may run while extended memory is paged in
BC00-BD00	Interrupt vector table
BD01-C4FF	The editor
C500-C5FF	Integer variables and array descriptors
C600 upwards	Your program, then the named variables, then the normal arrays

Address (hex)	Contents
down from FF40	Z80 stack
Banks 1, 3, 4, 6 (128K)	FAR arrays and maps, paged in at C000-FFFF when used
Bank 7 C000-C65F (128K)	Music notes for 3 channels
Bank 7 C660-CA45 (128K)	Tune compiler and effect parser, run in place

Integer variable A is at hex C582, B at C584, and so on: `DPEEK(50562 + 2*K)` reads variable number K (0 = A). The time-critical FastZX code runs in uncontended RAM (bank 2 on the 128K) for full speed. The code block on tape starts at 7300 hex so the tables, messages and font in low memory load with it; it is still started at 32768. The interrupt vector table must lie below C000, and so must every routine that can run while an extended-memory bank is paged in at C000. The build checks these rules, that no variable overlaps the interrupt jump at 7F7F, and that the low-memory tables and the bank-7 module fit, and stops with an error otherwise.

Entry points

Address	Use
32768	Cold start (empty program)
32771	Return to FastZX keeping the program (after BYE)
32774	Start a compiled program
32777	Start the stored program immediately (used by `SAVE "name" RUN`)

128K details

At start-up FastZX pages in the 48K BASIC ROM and memory bank 0 by setting port 7FFD (keeping the other bits), checks whether the 128K paging works, and restores the original value on BYE. On a 128K it also switches the Z80 to interrupt mode 2: 50 times a second its own routine updates the music and sound effect, then continues into the ROM's normal interrupt routine (clock and keyboard). The vector table is at BC00 hex, below C000, so bank switching never affects it. BYE silences the AY and returns to interrupt mode 1. FAR

arrays live in banks 1, 3, 4 and 6: a FAR address is a 16-bit offset whose top two bits choose the bank and whose other 14 bits give the address in the window at C000. Each access switches the bank in with interrupts briefly disabled and switches bank 0 back, so the stack and program at the top of memory are never hidden while in use; block copies (SCRSAVE, SCRLOAD, DIM) go through a buffer in low memory. Of bank 7, only the second screen area (C000-DAFF) is used: the music notes, and the tune compiler and effect parser, which run in place there on a small stack in low memory (they only read their text from the string work area and write notes into bank 7). The interrupt routine reads each note from bank 7 with interrupts off and without using the stack while bank 7 is paged in. On a 48K MUSIC and SFX do nothing, as before. FastZX uses ROM routines for PRINT, CLS, PLOT, DRAW, POINT, BEEP and the tape; everything else is its own code.

Program format

Each stored line is: line number (2 bytes, kept equal to its position), length (1 byte), indentation (1 byte), tokens, end-of-line byte 13. Numbers are stored as byte 1 followed by 2 bytes; labels and procedure names follow their token as plain letters. A named variable is stored as its kind (2 number, 3 string, 4 array), its number (two letters, filled in when the program runs), the name's length as a letter, and the name; every byte after the kind is a letter, so the routines that scan program lines treat it like ordinary text. SAVE writes a standard CODE file starting at 50688 (hex C600) with FA57 hex as its second parameter; LOAD accepts only files with that marker, and always loads them at the current program start, whatever address they were saved from. Programs saved by versions 1.0 to 3.0 cannot be loaded, because the format changed. Programs saved by 4.0, 4.1, 4.2 and 4.3 load and run unchanged in this version: new keywords are always added without changing any existing keyword code. FAR arrays are not saved with a program; they are created when it runs.

Machine code

`CALL addr` and `USR(addr)` run your own machine code. It must keep the IY register, may change all others, and ends with RET; USR returns the BC register. Put machine code in a BYTE array (`DIM M() BYTE = ...`, then `CALL ADR(M)`). Remember that arrays move when the program is edited.

Limitations

- No floating point, no long variable names, no DATA/READ, no ON GOTO, no string comparison with `<` or `>`, no string parameters.

- One-dimensional arrays only.
- Sprites must lie fully on the screen; there is no clipping at the edges.
- Arithmetic overflow wraps silently. A FOR loop whose counter would pass 32767 ends instead of wrapping.
- Not yet verified on physical hardware: everything was tested in a Z80 emulator in which the ROM routines are simulated. Please report anything that behaves differently on your 128K.

Rebuilding

`fastzx.asm` is the complete source in standard Z80 syntax (sjasmplus-compatible). `fastzx_source_and_tools.zip` holds the same source in four parts, the build script (`python3 build.py` makes a new fastzx.tap), the compiler, the demo programs as `.bas` text files, and the test scripts with the emulator used to check everything.

20. Test programs and checklist

Language self-test

Type or load this program (it is also in the archive as `demos/langtest.bas`) and RUN it. Every line prints its result followed by the expected value in brackets; the two must match.

```
REM FastZX language self-test: every line shows the expected result in
brackets
PRINT 1+2*3;" [7]"
PRINT -7/2;" ";-7 MOD 2;" [-3 -1]"
PRINT 3<5;5<3;NOT 0;6 AND 3;6 OR 3;6 XOR 3;" [101275]"
PRINT $FF;" ";32767+1;" [255 -32768]"
DIM A(10)
FOR I = 0 TO 10: A(I) = I * I: NEXT I
S = 0: FOR I = 10 TO 0 STEP -2: S = S + A(I): NEXT I: PRINT S;" [220]"
FOR I = 5 TO 1: PRINT "never": NEXT I
W = 0: WHILE W < 3: W = W + 1: WEND: PRINT W;" [3]"
X = 0: REPEAT: X = X + 3: UNTIL X > 10: PRINT X;" [12]"
FOR I = 1 TO 3
  IF I = 2 THEN
    PRINT "two ";
  ELSE
    PRINT I;" ";
```

```
  ENDIF
NEXT I: PRINT "[1 two 3]"
N$ = "World": M$ = "Hello, " + N$ + CHR$(33): PRINT M$;LEN(M$);" [Hello,
World!13]"
PRINT MID$(M$,8,5);" ";STR$(-12)+"x";" ";ASC("A");" ";VAL("-42")+1;" [World
-12x 65 -41]"
DIM P() = 2, 3, 5, 7, 11: DIM B() BYTE = 200, 100: PRINT P(4);"
";PEEK(ADR(B)+1);" [11 100]"
GOSUB sub: PRINT "[back]"
PRINT FN SQ(9);" ";FN FIB(10);" [81 55]"
EXEC DIVIDE(17, 5): PRINT Q;" ";R;" [3 2]"
END
sub:
  PRINT "in sub ";: RETURN
FUNC SQ(N): RETURN N * N: ENDFUNC
FUNC FIB(N)
  IF N < 2 THEN RETURN N
  RETURN FN FIB(N - 1) + FN FIB(N - 2)
ENDFUNC
PROC DIVIDE(A, B)
  Q = A / B: R = A MOD B
ENDPROC
```

The GOSUB line prints `in sub [back]`. The same program compiled with fastzxc.py must print exactly the same.

Speed comparison with Sinclair BASIC

In FastZX:

```
T = TIME
FOR I = 1 TO 1000: A = I * 2 + 3: NEXT I
PRINT TIME - T;" frames"
```

In Sinclair BASIC (after BYE, or on a fresh machine), with line numbers:

```
10 POKE 23672,0: POKE 23673,0
20 FOR I=1 TO 1000: LET A=I*2+3: NEXT I
30 PRINT PEEK 23672+256*PEEK 23673;" frames"
```

50 frames = 1 second.

Checklist for version 4.0 on a real 128K

- [ ] fastzx.tap loads from the Tape Loader and shows the editor with the banner and free memory
- [ ] Typing, ENTER (split with auto-indent), DELETE (and joining lines), cursor keys, CAPS+3/4 page, CAPS+9 delete line all work
- [ ] Leaving a line tidies it (capital keywords, regular spacing) and keeps the indentation
- [ ] EDIT opens the command prompt; RUN, LIST, NEW and an empty ENTER behave as described
- [ ] An error opens the editor at the failing line
- [ ] The language self-test prints matching values
- [ ] Labels: GOTO and GOSUB to a name work; a wrong name gives Label not found
- [ ] DIM with values, DIM A() auto size, BYTE arrays and continuation lines work
- [ ] JOY works with a Kempston joystick (JOYSTICK 0) and with Sinclair 1 / Sinclair 2 (keys or a joystick in the +2 ports)
- [ ] SPRCOL colours a sprite and the colours are restored behind it
- [ ] The game demo: menu choice, movement, coloured sprites, scrolling stars, collisions
- [ ] The sprites demo: 8 masked coloured sprites move without damaging the background
- [ ] SAVE "test", NEW, LOAD "test" brings the program back into the editor
- [ ] SAVE "test" RUN makes a tape that loads from the Tape Loader and starts by itself; game.tap and sprites.tap do the same
- [ ] BYE returns to BASIC and RANDOMIZE USR 32771 comes back with the program

Version 4.1 additions:

- [ ] The banner at the bottom of the editor shows "+64K FAR free" on the 128K (and only "free" on a 48K)
- [ ] `PRINT FARFREE` gives 32767 after RUN with no FAR arrays
- [ ] A large FAR array (e.g. `DIM M(25000) FAR`) can be filled and read back correctly
- [ ] The screens demo stores three screens and switches between them instantly with keys 1-3
- [ ] BYE returns safely to 128 BASIC after using FAR arrays
- [ ] The same programs still run on a 48K Spectrum (FAR arrays then use normal memory)

Version 4.2 additions:

- [ ] Editor: moving the cursor down past the bottom of the screen scrolls one line at a time and shows the right lines (the bug from your video), also when scrolling back up
- [ ] The sound demo plays Ode to Joy on two channels while the star moves and the frame counter runs
- [ ] Keys 1-8 in the sound demo play the eight effects without stopping the music on channels A and B
- [ ] M in the sound demo stops and restarts the music
- [ ] The game demo plays its tune; a collision sounds the explosion effect
- [ ] `MUSIC STOP`, the end of a program and an error all leave the AY silent
- [ ] BYE returns to BASIC with normal interrupts (the 128K editor's keyboard and clock work)
- [ ] sound.tap loads from the Tape Loader and starts by itself

Version 4.3 additions:

- [ ] The editor starts in 51 columns and the small font is readable on your TV or monitor
- [ ] EXTEND (CAPS SHIFT + SYMBOL SHIFT) switches to 32 columns and back; the cursor is visible in both
- [ ] Typing a FOR, IF ... THEN, WHILE or PROC block flush left indents it automatically, and NEXT / ENDIF / WEND / ENDPROC line up with their opener
- [ ] The tiles demo: the maze is built, the view scrolls when you walk near the edge, walls stop you, coins disappear with a sound and the counter goes up
- [ ] tiles.tap loads from the Tape Loader and starts by itself
- [ ] The cookbook programs (section 15) work as described: jumping onto the platforms, shooting the enemy, the menu, the reaction timer, the high-score table
- [ ] Programs saved with 4.2 load and run in 4.3

Sprite update (after 4.3):

- [ ] The sprites demo: the eight sprites stay visible nearly all the time; only now and then one blinks (before this update most of them were missing in most frames)
- [ ] The game and tiles demos: the player and enemy sprites do not flicker noticeably
- [ ] Overlapping sprites in the sprites demo still pass over each other cleanly, with no leftover pieces of background

Memory update (about 2 KB more for programs):

- [ ] FastZX loads from the Tape Loader (three blocks: loader, code, and a short block that briefly shows as a pattern at the top of the screen)

- [ ] The banner reports about 16100 bytes free on a 128K
- [ ] Music and sound effects work as before (the sound demo, keys 1-8, M)
- [ ] After BYE, 128 BASIC works normally: edit and run a BASIC line, then re-enter FastZX with RANDOMIZE USR 32771 and play music again
- [ ] On a +2A or +3: BYE, then CAT or LOAD from disk still work
- [ ] A program saved with 4.3 loads and runs
- [ ] sound.tap and game.tap (stand-alone) load and play their music

The checklists below are from earlier versions and are kept with their ticks. Some items mention line numbers, which version 4.0 no longer uses.

Earlier checklists (versions 1.0 to 3.0)

- [ ] Tape loads from the 128K Tape Loader and shows the banner
- [ ] Typing, DELETE, caps lock and symbol shift keywords work in the editor
- [ ] LIST, EDIT, line replace and line delete work
- [ ] The language test prints the expected values
- [ ] BREAK stops `10 GOTO 10` and an endless `FOR` loop
- [ ] PLOT, DRAW, POINT and colours work in the graphics test
- [ ] BEEP sounds; SOUND plays a tone through the AY chip
- [ ] SAVE "test", NEW, LOAD "test" restores the program
- [ ] The three demos on the tape load and run
- [ ] BYE returns to BASIC and RANDOMIZE USR 32771 comes back with the program intact

Version 2.0 additions:

- [ ] The procs demo prints 7 Hanoi moves, the Fibonacci numbers, GCD 21 and 7! = 5040
- [ ] The game demo: sprites move smoothly with Q/A/O/P, the star strip scrolls, a collision flashes the border
- [ ] KEY() reacts to two keys held together (e.g. Q and P)
- [ ] PUTCH, PUTS, FILLATTR and FILLBOX draw in the right cells (column first, row second)
- [ ] SCROLL and SCROLLC move only the chosen rectangle, in all four directions

Version 3.0 additions:

- [ ] The sprites demo: 8 masked sprites bounce over the stripes, the background is clean behind them, overlapping sprites look right
- [ ] SPRDEF and SPRMASK with sizes 8,8 / 8,16 / 16,8 / 16,16 draw the right shape

- [ ] SAVE "test" RUN writes a tape that loads from the 128K Tape Loader and starts the program by itself
- [ ] game.tap and sprites.tap load and run from the Tape Loader without FastZX
- [ ] A program compiled with fastzxc.py loads from the Tape Loader and gives the same results as in FastZX